

Scenario Summary			
	Current Values:	Avg performance	Best Performance
Changing Cells:			
\$B\$2	70	80	95
\$B\$3	20	25	30
\$B\$4	10	20	25
Result Cells:			
Total	100	125	150
per	33.33333333	41.66666667	50

Notes: Current Values column represents values of changing cells at time Scenario Summary Report was created. Changing cells for each scenario are highlighted in gray.

Aim:

Creating data frames from various types of files (csv/json etc) and performing various basic data pre-processing functions.

Program / Code:

```
import pandas as pd
df = pd.read_csv("Employees.csv")
print("----- Original Data -----")
print(df)

# 1. handling missing values
df_filled = df.copy()
df_filled['Designation'] =
df_filled['Designation'].fillna("Not Assigned")
print(df_filled)
df_dropped = df.dropna(subset=['Designation'])
print(df_dropped)

# 2. Filter Salary column to show employees with
salary higher than 40K
high_salary = df[df['Salary'] > 40000]
print(high_salary)

# 3. Filter Designation to show only Marketing
Employees
marketing_emp = df[df['Department'] == 'Marketing']
print(marketing_emp)

# 4. Sort the employee names in forward and reverse
order
```

```
sorted_forward = sorted(df['Emp_Name'])
print(sorted_forward)
sorted_reverse = sorted(df['Emp_Name'], reverse=True)
print(sorted_reverse)

# 5. Display only Tech employees
tech_emp = df[df['Department'] == 'Tech']
print(tech_emp)
# 6. Find the Average salary of all departments
dept_avg_salary =
df.groupby('Department')['Salary'].mean()
print(dept_avg_salary)

# 7. Find the count of HR employees
hr_count = df[df['Department'] ==
'HR']['Emp_Name'].count()
print(hr_count)
# 8. Find the designation wise highest and lowest
salary
designation_max_salary =
df.groupby('Designation')['Salary'].max()
print(designation_max_salary)
designation_min_salary =
df.groupby('Designation')['Salary'].min()
print(designation_min_salary)
```

Output / Result:

```
----- Original Data -----
   Emp_Name  Department Designation  Salary
0  Amit Sharma      Tech      Manager  85000
1  Priya Singh  Accounting  Team Lead  45000
2  Rahul Verma      HR      Executive  22000
3  Sneha Gupta  Marketing      Manager  92000
4  Vikram Rao      Tech      NaN      55000
5  Anjali Nair      HR      Team Lead  38000
6  Karan Mehta  Marketing  Executive  27000
7  Divya Iyer  Accounting      Manager  68000
8  Rohan Kapoor      Tech      Team Lead  47000
9  Neha Joshi  Marketing      NaN      31000
10 Arjun Reddy      HR      Manager  75000
11 Pooja Desai      Tech      Executive  19000
12 Manish Yadav  Accounting      NaN      44000
13 Kavita Menon  Marketing  Team Lead  52000
14 Suresh Pillai      HR      NaN      26000
```

```
----- Lowest Salary by Designation -----
Designation
Executive      19000
Manager        68000
Team Lead      38000
Name: Salary, dtype: int64
```

```
----- Tech Department Employees -----
   Emp_Name  Department Designation  Salary
0  Amit Sharma      Tech      Manager  85000
4  Vikram Rao      Tech      NaN      55000
8  Rohan Kapoor      Tech      Team Lead  47000
11 Pooja Desai      Tech      Executive  19000
```

```
----- Highest Salary by Designation -----
Designation
Executive      27000
Manager        92000
Team Lead      52000
Name: Salary, dtype: int64
```

```
----- Lowest Salary by Designation -----
Designation
Executive      19000
Manager        68000
Team Lead      38000
Name: Salary, dtype: int64
```

```
----- Average Salary by Department -----
Department
Accounting      52333.333333
HR               40250.000000
Marketing        50500.000000
Tech             51500.000000
Name: Salary, dtype: float64
```

```
   Emp_Name  Department Designation  Salary
3  Sneha Gupta  Marketing      Manager  92000
6  Karan Mehta  Marketing  Executive  27000
9  Neha Joshi  Marketing      NaN      31000
13 Kavita Menon  Marketing  Team Lead  52000
```

Aim:

Practical on feature scaling & Feature Dummification.

A. Use the wine dataset to apply min max scaler and standard (z-score) scaler on the dataset.

B. Use iris data set and apply label encoding and one hot encoding on the species column.

Program / Code (A):

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler
df = pd.read_csv('wine.csv',header=None,usecols=[0,1,2])
df.columns = ['Class','Alcohol_Level','Malic_Acid']
print(df)

#Min-Max Scaler
mm = MinMaxScaler()
df[['Class','Alcohol_Level','Malic_Acid']] = mm.fit_transform(df[['Class','Alcohol_Level','Malic_Acid']])
print(df)

#Standard Scaler
ss = StandardScaler()
df[['Class','Alcohol_Level','Malic_Acid']] = ss.fit_transform(df[['Class','Alcohol_Level','Malic_Acid']])
print(df)
```

Program / Code (B):

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
df = pd.read_csv('iris-id.csv')
print(df)

#label Encoder
le = LabelEncoder()
df['species_ID'] = le.fit_transform(df['species'])
print(df)

#One Hot Encoding
dummy = pd.get_dummies(df,columns=['species'],drop_first=False,dummy_na=False)
print(dummy[['species_setosa','species_versicolor','species_virginica']])
```

Output / Result:

	Class	Alcohol_Level	Malic_Acid
0	14.23	1.71	2.43
1	13.20	1.78	2.14
2	13.16	2.36	2.67
3	14.37	1.95	2.50
4	13.24	2.59	2.87
..	...	...	...
173	13.71	5.65	2.45
174	13.40	3.91	2.48
175	13.27	4.28	2.26
176	13.17	2.59	2.37
177	14.13	4.10	2.74

	Class	Alcohol_Level	Malic_Acid
0	1.518613	-0.562250	0.232053
1	0.246290	-0.499413	-0.827996
2	0.196879	0.021231	1.109334
3	1.691550	-0.346811	0.487926
4	0.295700	0.227694	1.840403
..	...	...	...
173	0.876275	2.974543	0.305159
174	0.493343	1.412609	0.414820
175	0.332758	1.744744	-0.389355
176	0.209232	0.227694	0.012732
177	1.395086	1.583165	1.365208

	Class	Alcohol_Level	Malic_Acid
0	0.842105	0.191700	0.572193
1	0.571053	0.205534	0.417112
2	0.560526	0.320158	0.700535
3	0.878947	0.239130	0.609626
4	0.581579	0.365613	0.807487
..	...	...	...
173	0.705263	0.970356	0.582888
174	0.623684	0.626482	0.598930
175	0.589474	0.699605	0.481283
176	0.563158	0.365613	0.540107
177	0.815789	0.664032	0.737968

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
..	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

	sepal_length	sepal_width	petal_length	petal_width	species	species_ID
0	5.1	3.5	1.4	0.2	setosa	0
1	4.9	3.0	1.4	0.2	setosa	0
2	4.7	3.2	1.3	0.2	setosa	0
3	4.6	3.1	1.5	0.2	setosa	0
4	5.0	3.6	1.4	0.2	setosa	0
..	...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica	2
146	6.3	2.5	5.0	1.9	virginica	2
147	6.5	3.0	5.2	2.0	virginica	2
148	6.2	3.4	5.4	2.3	virginica	2
149	5.9	3.0	5.1	1.8	virginica	2

	species_setosa	species_versicolor	species_virginica
0	True	False	False
1	True	False	False
2	True	False	False
3	True	False	False
4	True	False	False
..	...	...	...
145	False	False	True
146	False	False	True
147	False	False	True
148	False	False	True
149	False	False	True

Aim:

Practical on Hypothesis Testing (Z-Test and T-test).

Program / Code:

```
import numpy as np
from scipy import stats
###Q1.
print("H0 : mean age >= 19")
print("H1 : mean age < 19")
x = 18.1
u = 19
psd = 2.1
n = 40
s1 = 0.05
z = (x-u)/(psd/np.sqrt(n))
p = stats.norm.cdf(z)
print(p)
if p > s1:
    print("Null Hypothesis Wins\n")
else :
    print("Alternate Hypothesis Wins\n")

###Q2.
print("H0 : Newspaper Price == 100")
print("H1 : Newspaper Price != 100")
x = 95
u = 100
ssd = 20
n = 12
dof = n-1
s1 = 0.01
t = (x-u)/(ssd/np.sqrt(n))
t = 2*(stats.t.sf(t,dof))
print(t)
if t > s1:
    print("Null Hypothesis Wins\n")
else :
    print("Alternate Hypothesis Wins\n")

###Q3.
print("H0 : Glucose Level == 100")
print("H1 : Glucose Level != 100")
x = 140
u = 100
psd = 15
n = 30
s1 = 0.05
z = (x-u)/(psd/np.sqrt(n))
p = 2*(stats.norm.sf(z))
print(p)
if p > s1:
    print("Null Hypothesis Wins\n")
else :
    print("Alternate Hypothesis Wins\n")
```

```
###Q4.
print("H0 : Screen Time <= 4.5")
print("H1 : Screen Time > 4.5")
x = 4.75
u = 4.5
psd = 2.0
n = 15
dof = n-1
s1 = 0.05
z = (x-u)/(psd/np.sqrt(n))
p = stats.t.sf(z,dof)
print(p)
if p > s1:
    print("Null Hypothesis Wins\n")
else :
    print("Alternate Hypothesis Wins\n")

###Q5.
print("H0 : Sleep Time < 7")
print("H1 : Sleep Time >= 7")
x = 7.24
u = 7
psd = 1.93
n = 22
dof = n-1
s1 = 0.05
z = (x-u)/(psd/np.sqrt(n))
p = stats.t.sf(z,dof)
print(p)
if p > s1:
    print("Null Hypothesis Wins\n")
else :
    print("Alternate Hypothesis Wins\n")

###Q6.
print("H0 : Tire Average => 50000")
print("H1 : Sleep Time < 50000")
x = 46500
u = 50000
psd = 8000
n = 28
dof = n-1
s1 = 0.05
z = (x-u)/(psd/np.sqrt(n))
p = stats.norm.cdf(z)
print(p)
if p > s1:
    print("Null Hypothesis Wins\n")
else :
    print("Alternate Hypothesis Wins\n")
```

Output / Result:

```
H0 : mean age >= 19
H1 : mean age < 19
0.0033588522877313273
Alternate Hypothesis Wins

H0 : Newspaper Price == 100
H1 : Newspaper Price != 100
1.5950304754922338
Null Hypothesis Wins

H0 : Glucose Level == 100
H1 : Glucose Level != 100
2.574266663697655e-48
Alternate Hypothesis Wins
```

```
H0 : Screen Time <= 4.5
H1 : Screen Time > 4.5
0.317892101098044
Null Hypothesis Wins

H0 : Sleep Time < 7
H1 : Sleep Time >= 7
0.2829637224614778
Null Hypothesis Wins

H0 : Tire Average => 50000
H1 : Sleep Time < 50000
0.010305579572800304
Alternate Hypothesis Wins
```

Aim:

Practical on Hypothesis Testing (Chi-Square Testing).

Program / Code (1):

```
import numpy as np
from scipy import stats

observed = np.array([12,10,8,10,11,9])
print("Observed frequency : ",observed)
faces = 6
expected = np.array([observed.sum()/faces]*faces)
print("Expected frequency : ",expected)
dof = faces-1

chi2 = sum(((observed-expected)**2)/expected)
print("Chisquare Value : ",chi2)
p_value = stats.chi2.sf(chi2,dof)
print("Probability : ",p_value)

chi2,p_value = stats.chisquare(observed,expected)
print("Built-In Formula ",p_value)
```

Program / Code (2):

```
import numpy as np
from scipy import stats

observed = np.array([[25,30,15],[30,25,40]])
print("Array : \n",observed)
row_sum = observed.sum(axis=1)
col_sum = observed.sum(axis=0)
grand_total = observed.sum()
expected = np.outer(row_sum,col_sum)/grand_total
print(expected)
dof = (observed.shape[0]-1)*(observed.shape[1]-1)

chi2 = (((observed-expected)**2)/expected).sum()
print("Chisquare Value : ",chi2)
p_value = stats.chi2.sf(chi2,dof)
print("Probability : ",p_value)

chi2,p_value,dof,expected = stats.chi2_contingency(observed)
print("Built-In Formula ",p_value)
```

Output / Result:

```
1.
Observed frequency : [12 10 8 10 11 9]
Expected frequency : [10. 10. 10. 10. 10. 10.]
Chisquare Value : 1.0
Probability : 0.9625657732472964
Built-In Formula 0.9625657732472964

2.
Array :
[[25 30 15]
 [30 25 40]]
[[23.33333333 23.33333333 23.33333333]
 [31.66666667 31.66666667 31.66666667]]
Chisquare Value : 8.684210526315788
Probability : 0.013009111750454501
Built-In Formula 0.013009111750454501
```

Aim:

Practical on ANOVA.

Program / Code:

```
import numpy as np
import pandas as pd
from scipy import stats

df = pd.read_csv("College.csv")
Vesasc = np.array(df['Vesasc'])
Nga = np.array(df['Nga'])
Bt = np.array(df['Bt'])
Vesasc_mean = Vesasc.mean()
Nga_mean = Nga.mean()
Bt_mean = Bt.mean()
n = 5
k = 3
N = 15
Total_mean = np.array([Vesasc_mean, Nga_mean, Bt_mean]).mean()
a = (Vesasc_mean - Total_mean)**2
b = (Nga_mean - Total_mean)**2
c = (Bt_mean - Total_mean)**2
total_sum_mean = n*(a+b+c)
ssb = total_sum_mean/(k-1)
ssw = (sum((Vesasc - Vesasc_mean)**2) + sum((Nga - Nga_mean)**2) + sum((Bt - Bt_mean)**2)) / (N-k)
F = ssb/ssw
print("Variance between the group :-" , ssb)
print("Variance within the group :-" , ssw)
print("F statistics score :- ", F)
print("P-value :- ", stats.f.sf(F, (k-1), (N-k)))

F, p_value = stats.f_oneway(Vesasc, Nga, Bt)
print("F Statistics : ", F, "\nP Value : ", p_value)
```

Output / Result:

```
VESASC : [87 79 90 77 92]
NGA : [ 0 85 0 83 87]
BT : [78 87 82 89 75]
Number Of Groups(k) : 3
Number Of Samples Per Group(n) : 5
Variance between the group :- 1781.0666666666667
Variance within the group :- 749.5666666666666
F statistics score :- 2.3761284297594174
P-value :- 0.13509756961139197
F Statistics : 2.376128429759415
P Value : 0.13509756961139216
```

Aim:

Practical on Linear Regression.

Program / Code:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import LabelEncoder

df = pd.read_csv("Housing.csv")
print(df.head)
le = LabelEncoder()
df["mainroad"] = le.fit_transform(df["mainroad"])
df["guestroom"] = le.fit_transform(df["guestroom"])
df["basement"] = le.fit_transform(df["basement"])
df["hotwaterheating"] = le.fit_transform(df["hotwaterheating"])
df["airconditioning"] = le.fit_transform(df["airconditioning"])
df["prefarea"] = le.fit_transform(df["prefarea"])
df["furnishingstatus"] = le.fit_transform(df["furnishingstatus"])
X = df[["area"]].values
Y = df[["price"]].values

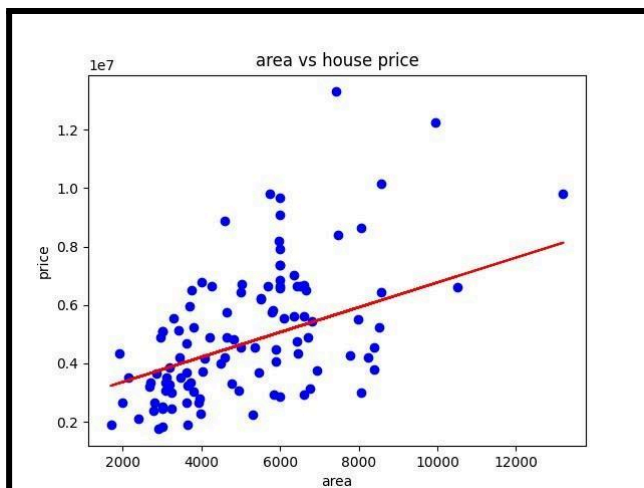
X_train,X_test,Y_train,Y_test = train_test_split(X,Y,test_size=0.2,random_state=42)
# print(X_train)
reg = LinearRegression()
reg.fit(X_train,Y_train)
Y_predicted = reg.predict(X_test)
# print(Y_predicted)
# error
mse = mean_squared_error(Y_test,Y_predicted)
r2 = r2_score(Y_test,Y_predicted)
print("mean_squared_error",mse)
print("R Squared",r2)
house_price = reg.predict([[5000]])
print(house_price)

# Plotting
plt.scatter(X_test,Y_test,color = "blue")
plt.plot(X_test,Y_predicted,color= "red")
plt.xlabel("area")
plt.ylabel("price")
plt.title("area vs house price")
plt.show()
```

Output / Result:

```
<bound method NDFrame.head of
ng prefarea furnishingstatus price area bedrooms bathrooms stories mainroad ... basement hotwaterheating airconditioning parki
0 13300000 7420 4 2 3 yes ... no no yes 2 yes furnished
1 12250000 8960 4 4 4 yes ... no no yes 3 no furnished
2 12250000 9960 3 2 2 yes ... yes no no 2 yes semi-furnished
3 12215000 7500 4 2 2 yes ... yes no yes 3 yes furnished
4 11410000 7420 4 1 2 yes ... yes no yes 2 no furnished
... ..
540 1820000 3000 2 1 1 yes ... yes no no 2 no unfurnished
541 1767150 2400 3 1 1 no ... no no 0 no semi-furnished
542 1750000 3620 2 1 1 yes ... no no 0 no unfurnished
543 1750000 2910 3 1 1 no ... no no 0 no furnished
544 1750000 3850 3 1 2 yes ... no no 0 no unfurnished

[545 rows x 13 columns]
mean_squared_error 3675286604768.185
R Squared 0.27287851871974644
[[4640903.47365326]]
```

Graph:

Aim:

Practical on KMeans Clustering And Decision Tree.

Program / Code:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.cluster import KMeans

df = pd.read_csv('income.csv')
mm = MinMaxScaler()
df['Age'] = mm.fit_transform(df[['Age']])
df['Income($)' ] = mm.fit_transform(df[['Income($)']])
print(df)

km = KMeans(n_clusters=3)
X = df[['Age', 'Income($)']]
y_predict = km.fit_predict(X)
print(y_predict)
df['clusters'] = y_predict
print(df)

centroids = km.cluster_centers_
print(centroids)
df1 = df[df['clusters']==0]
df2 = df[df['clusters']==1]
df3 = df[df['clusters']==2]
```

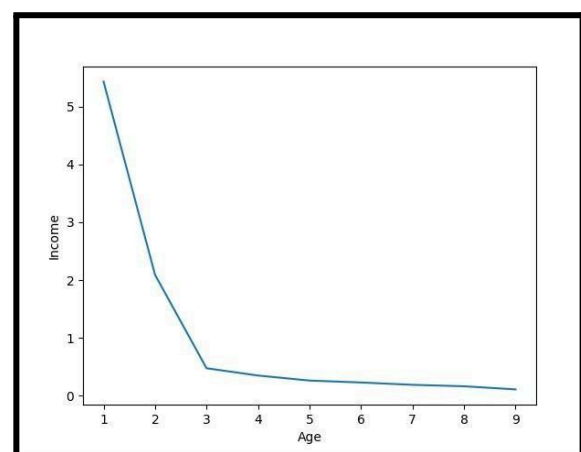
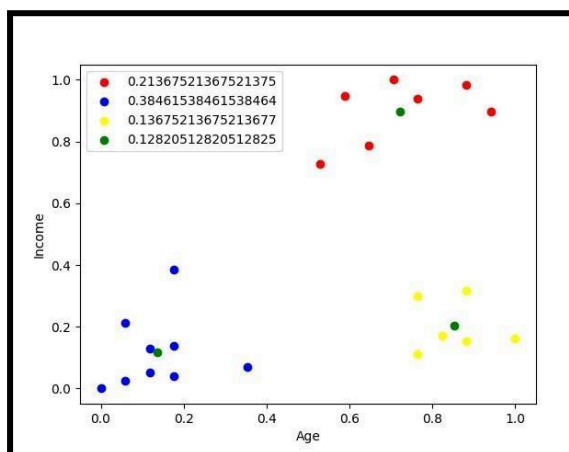
```
plt.scatter(df1['Age'], df1['Income($)'], color='red')
plt.scatter(df2['Age'], df2['Income($)'],
color='blue')
plt.scatter(df3['Age'], df3['Income($)'],
color='yellow')
plt.scatter(centroids[:,0], centroids[:,1],
color='green')
plt.xlabel('Age')
plt.ylabel('Income')
plt.legend(df['Income($)'])
plt.show()

#elbow method to find best numbers of clusters
sse = []
rng = range(1,10)
for k in rng:
    km = KMeans(n_clusters=k)
    km.fit(X)
    sse.append(km.inertia_)
plt.xlabel('Age')
plt.ylabel('Income')
plt.plot(rng,sse)
plt.show()
```

Output / Result:

	Name	Age	Income(\$)
0	Rob	0.058824	0.213675
1	Michael	0.176471	0.384615
2	Mohan	0.176471	0.136752
3	Ismail	0.117647	0.128205
4	Kory	0.941176	0.897436
5	Gautam	0.764706	0.940171
6	David	0.882353	0.982906
7	Andrea	0.705882	1.000000
8	Brad	0.588235	0.948718
9	Angelina	0.529412	0.726496
10	Donald	0.647059	0.786325
11	Tom	0.000000	0.000000
12	Arnold	0.058824	0.025641
13	Jared	0.117647	0.051282
14	Stark	0.176471	0.038462
15	Ranbir	0.352941	0.068376
16	Dipika	0.823529	0.170940
17	Priyanka	0.882353	0.153846
18	Nick	1.000000	0.162393
19	Alia	0.764706	0.299145
20	Sid	0.882353	0.316239
21	Abdul	0.764706	0.111111

	Name	Age	Income(\$)	clusters
0	Rob	0.058824	0.213675	1
1	Michael	0.176471	0.384615	1
2	Mohan	0.176471	0.136752	1
3	Ismail	0.117647	0.128205	1
4	Kory	0.941176	0.897436	0
5	Gautam	0.764706	0.940171	0
6	David	0.882353	0.982906	0
7	Andrea	0.705882	1.000000	0
8	Brad	0.588235	0.948718	0
9	Angelina	0.529412	0.726496	0
10	Donald	0.647059	0.786325	0
11	Tom	0.000000	0.000000	1
12	Arnold	0.058824	0.025641	1
13	Jared	0.117647	0.051282	1
14	Stark	0.176471	0.038462	1
15	Ranbir	0.352941	0.068376	1
16	Dipika	0.823529	0.170940	2
17	Priyanka	0.882353	0.153846	2
18	Nick	1.000000	0.162393	2
19	Alia	0.764706	0.299145	2
20	Sid	0.882353	0.316239	2
21	Abdul	0.764706	0.111111	2

Graph Plotting:

Aim:

Logistic Regression and PCA

Program / Code:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
from sklearn.metrics import confusion_matrix

df = pd.read_csv('titanic.csv')
print(df.head())

df = df.drop(['PassengerId', 'Name', 'Ticket', 'Cabin'], axis=1)
df['Age'] = df['Age'].fillna(df['Age'].median())

le = LabelEncoder()
df['Sex'] = le.fit_transform(df['Sex'])
df['Embarked'] = le.fit_transform(df['Embarked'])
X = df.drop(['Survived'], axis=1)
Y = df['Survived']
print(X)

s = StandardScaler()
X = s.fit_transform(X)
pca = PCA(n_components=4)
X_pca = pca.fit_transform(X)
X_train, X_test, Y_train, Y_test = train_test_split(X_pca, Y,
                                                    random_state=42)

model = LogisticRegression()
model.fit(X_train, Y_train)
y_prediction = model.predict(X_test)

#PCA
cm = confusion_matrix(Y_test, y_prediction)
print("Centroids : ",cm)
plt.figure()
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Not Survived', 'Survived'],
            yticklabels=['Not Survived', 'Survived'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```

Output / Result:

[Running] python -u "c:\College Codings\SEM V\DataScience\P8.py"

```
PassengerId Survived Pclass ... Fare Cabin Embarked
0      1      0      3 ... 7.2500 NaN      S
1      2      1      1 ... 71.2833 C85      C
2      3      1      3 ... 7.9250 NaN      S
3      4      1      1 ... 53.1000 C123     S
4      5      0      3 ... 8.0500 NaN      S
```

[5 rows x 12 columns]

```
Centroids : [[108 26]
             [ 25 64]]
```

